

# AWS Data Engineering Capstone Project

## Claims 360- Insurance Claims Data Lakehouse on AWS

### 1. Project Overview

#### Objective

The Claims 360 project involves designing and implementing a modern cloud-native Insurance Data Lakehouse architecture on AWS. The solution consolidates structured and semi-structured data from multiple legacy systems into a centralized analytics platform.

The project demonstrates the following capabilities:

- Batch data ingestion from relational and NoSQL sources
- Event-driven ETL orchestration using AWS Lambda and Glue
- Data Lakehouse implementation using Delta Lake on Amazon S3
- ACID transaction handling and schema evolution support
- Time-travel analytics and unified querying using Amazon Athena

The final solution enables insurance analysts to monitor the latest state of every claim while preserving the complete historical audit trail of all updates.

---

### 2. Business Problem Statement

Insurance companies commonly operate across multiple disconnected systems, each managing a different slice of the business:

- Policy details are maintained in relational databases
- Claim activities and audit logs are stored in NoSQL systems
- Historical claim tracking is difficult to implement consistently
- Data consistency issues arise frequently during updates
- Analytics teams struggle to generate unified business insights

## Existing Challenges

- No centralized analytics platform
- Difficult to track the full claim lifecycle history
- Lack of transactional consistency across systems
- Manual reconciliation required between source systems
- Schema changes break downstream pipelines
- Limited scalability for growing volumes of claim data

## Business Requirement

- Scalable AWS-based Data Lakehouse
- Integration of RDS and DynamoDB data
- Maintenance of historical claim versions
- Support for real-time analytics
- ACID-compliant storage
- Time-travel querying capabilities
- Automatic schema evolution handling

---

# 3. Solution Architecture

## Architecture Components

| AWS Service             | Purpose                                           |
|-------------------------|---------------------------------------------------|
| Amazon RDS (PostgreSQL) | Stores structured customer and policy data        |
| Amazon DynamoDB         | Stores claim audit logs and status updates        |
| Amazon S3               | Data Lake storage – Bronze and Silver layers      |
| AWS Lambda              | Event-driven trigger to initiate the ETL pipeline |
| AWS Glue (PySpark)      | Distributed ETL engine for data transformation    |
| AWS Glue Data Catalog   | Metadata management for Delta tables              |
| Amazon Athena           | Serverless SQL analytics and time-travel queries  |
| Delta Lake              | ACID-compliant Lakehouse storage format           |
| AWS IAM                 | Security, permissions, and least-privilege access |
| Amazon CloudWatch       | Logging and monitoring of pipeline runs           |

## 4. Dataset Details

### Source Datasets

#### 1. Customers Dataset (RDS)

| Column  | Description                      |
|---------|----------------------------------|
| cust_id | Unique customer identifier       |
| name    | Customer name                    |
| email   | Customer email address           |
| risk    | Risk category (Low / Med / High) |

#### 2. Policies Dataset (RDS)

| Column    | Description                    |
|-----------|--------------------------------|
| policy_id | Unique policy identifier       |
| cust_id   | Foreign key to customers table |
| type      | Insurance type (e.g., Auto)    |
| premium   | Policy premium amount          |

#### 3. ClaimStatusAudit Dataset (DynamoDB)

| Column     | Description                               |
|------------|-------------------------------------------|
| claim_id   | Claim identifier (Partition Key)          |
| updated_at | Timestamp of the update (Sort Key)        |
| status     | Claim status – Open / Under_Review / Paid |
| amount     | Amount associated with the claim          |

### Dataset Generation

The following Python script was used to generate the sample dataset files locally before uploading to S3.

```
import pandas as pd
import random
import uuid
from datetime import datetime, timedelta
import os

def generate_insurance_data(output_path):
    os.makedirs(output_path, exist_ok=True)
```

```

# RDS - Customers & Policies
custs = [
    {
        'cust_id': i,
        'name': f"Insured_{i}",
        'email': f"insured_{i}@example.com",
        'risk': random.choice(['Low', 'Med', 'High'])
    }
    for i in range(1, 51)
]

policies = [
    {
        'policy_id': f"POL_{100+i}",
        'cust_id': i,
        'type': 'Auto',
        'premium': random.randint(500, 2000)
    }
    for i in range(1, 51)
]

# DynamoDB - Claims Audit (multiple states per claim)
claims = []
for _ in range(100):
    c_id = f"CLM_{random.randint(1000, 1020)}"
    claims.append({
        "claim_id": c_id,
        "updated_at": (datetime.now() - timedelta(hours=random.randint(1,
100))).isoformat(),
        "status": random.choice(["Open", "Under_Review", "Paid"]),
        "amount": round(random.uniform(1000, 5000), 2)
    })

pd.DataFrame(custs).to_csv(os.path.join(output_path, "customers.csv"), index=False)
pd.DataFrame(policies).to_csv(os.path.join(output_path, "policies.csv"), index=False)
pd.DataFrame(claims).to_json(os.path.join(output_path, "claims_audit.json"),
orient="records")
print(f"Files generated at: {output_path}")

generate_insurance_data(r"C:\Users\sreevarshith\Documents\AWS\Claims_360_Project\Datasets
")

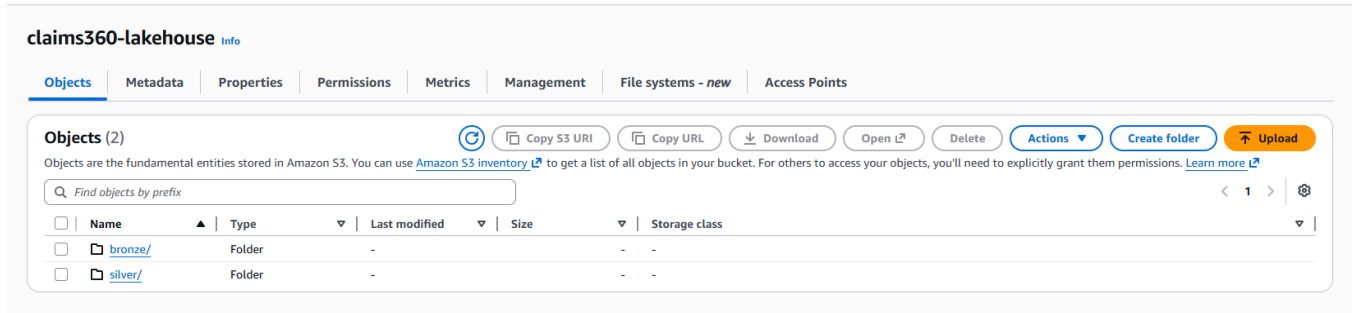
```

The script produces three output files:

| File Name         | Format | Records                 |
|-------------------|--------|-------------------------|
| customers.csv     | CSV    | 50 customer records     |
| policies.csv      | CSV    | 50 policy records       |
| claims_audit.json | JSON   | 100 claim audit entries |

## 5. AWS S3 Storage Setup

An Amazon S3 bucket was created to act as the central Data Lake, organized into Bronze and Silver layers to separate raw ingestion from processed Delta tables.



## Bucket Directory Structure

```
s3://claims360-lakehouse/
├── bronze/
│   ├── dynamodb/
│   │   └── claims_audit.json
│   └── rds/
│       ├── customers.csv
│       └── policies.csv
└── silver/
    ├── Claims-360-athena/
    └── delta/
        ├── insurance_claims/
        └── insurance_claims_history/
```

| Layer                      | Purpose                                |
|----------------------------|----------------------------------------|
| Bronze                     | Raw incoming files from source systems |
| Silver / delta             | Cleaned, transformed Delta Lake tables |
| Silver / Claims-360-athena | Athena query result storage            |

## 6. IAM Role Configuration

Two IAM roles were created to provide scoped access to the relevant AWS services.

**Claims\_GlueETLRole – attached to the AWS Glue job:**

| Policy                       | Type        |
|------------------------------|-------------|
| AmazonDynamoDBReadOnlyAccess | AWS Managed |
| AmazonRDSReadOnlyAccess      | AWS Managed |
| AmazonS3FullAccess           | AWS Managed |
| AWSGlueServiceRole           | AWS Managed |

A separate Lambda execution role was also created with the AWSLambdaBasicExecutionRole policy along with permissions to start Glue jobs.

Claims\_GlueETLRole

Claims\_GlueETLRole InfoDelete

Allows Glue to call AWS services on your behalf.

SummaryEdit

Creation date

April 30, 2026, 11:32 (UTC+05:30)

Last activity

17 days ago

ARN

arn:aws:iam::622238671639:role/Claims\_GlueETLRole

Maximum session duration

1 hour

PermissionsTrust relationshipsTagsLast AccessedRevoke sessions

Permissions policies (4) Info

You can attach up to 10 managed policies.

Filter by Type

All types

< 1 > ⚙

| <input type="checkbox"/> | Policy name ⓘ                | Type        | Attached entities |
|--------------------------|------------------------------|-------------|-------------------|
| <input type="checkbox"/> | AmazonDynamoDBReadOnlyAccess | AWS managed | 1                 |
| <input type="checkbox"/> | AmazonRDSReadOnlyAccess      | AWS managed | 1                 |
| <input type="checkbox"/> | AmazonS3FullAccess           | AWS managed | 3                 |
| <input type="checkbox"/> | AWSGlueServiceRole           | AWS managed | 1                 |

LambdaGlueTriggerRole

LambdaGlueTriggerRole InfoDelete

Allows Lambda functions to call AWS services on your behalf.

SummaryEdit

Creation date

April 30, 2026, 11:33 (UTC+05:30)

Last activity

17 days ago

ARN

arn:aws:iam::622238671639:role/LambdaGlueTriggerRole

Maximum session duration

1 hour

PermissionsTrust relationshipsTagsLast AccessedRevoke sessions

Permissions policies (2) Info

You can attach up to 10 managed policies.

Filter by Type

All types

< 1 > ⚙

| <input type="checkbox"/> | Policy name ⓘ               | Type        | Attached entities |
|--------------------------|-----------------------------|-------------|-------------------|
| <input type="checkbox"/> | AWSGlueConsoleFullAccess    | AWS managed | 1                 |
| <input type="checkbox"/> | AWSLambdaBasicExecutionRole | AWS managed | 4                 |

7. Amazon RDS Setup

Amazon RDS was configured to store structured customer and policy information in a PostgreSQL database.

## RDS Configuration

| Parameter      | Value        |
|----------------|--------------|
| DB Identifier  | insurance-db |
| Engine         | PostgreSQL   |
| Instance Class | db.t4g.micro |
| Region / AZ    | us-east-1c   |
| Database Name  | insurance_db |
| Status         | Available    |

insurance-db

Modify

Actions

Summary

**DB identifier**  
insurance-db  
**CPU**  
4.33%

**Status**  
Available  
**Class**  
db.t4g.micro

**Role**  
Instance  
**Current activity**  
0.00 sessions

**Engine**  
PostgreSQL  
**Region & AZ**  
us-east-1c

**Recommendations**  
2 Informational

Connectivity & security

Monitoring

Logs & events

Configuration

Zero-ETL integrations

Maintenance & backups

Data migrations

Tags

Recommendations

Connect using

Info

☒ Code snippets  
Use when connecting through SDK, APIs, or third-party tools including agents.

☐ CloudShell  
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☐ Endpoints  
Use when connecting through any IDE interface.

Internet access gateway

Disabled

IAM Authentication

Enabled

IAM authentication token

Get token

Programming language

PSQL (Windows)

Endpoint type

Instance endpoint

Authentication type

IAM Authentication

Connection steps

Follow the steps below to paste the code of each step in your tool and run the commands. The snippets dynamically reflect the authentication configuration.

```
1 curl -o global-bundle.pem https://truststore.pk1.rds.amazonaws.com/global/global-bundle.pem
```

PowerShell

## Security Group Inbound Rule

| Type       | Protocol | Port Range | Source    |
|------------|----------|------------|-----------|
| PostgreSQL | TCP      | 5432       | 0.0.0.0/0 |

sg-05e93ed94c77b6f9c - default

Actions

Details

**Security group name**  
default  
**Owner**  
622238671639

**Security group ID**  
sg-05e93ed94c77b6f9c  
**Inbound rules count**  
2 Permission entries

**Description**  
default VPC security group  
**Outbound rules count**  
1 Permission entry

**VPC ID**  
vpc-06fb8b565c73cd224

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (2)

Manage tags

Edit inbound rules

| Name | Security group rule ID | IP version | Type        | Protocol | Port range | Source                  | Description |
|------|------------------------|------------|-------------|----------|------------|-------------------------|-------------|
| -    | sg-0f575202d3d158b48   | IPv4       | PostgreSQL  | TCP      | 5432       | 0.0.0.0/0               | -           |
| -    | sg-06b4158502adce966   | -          | All traffic | All      | All        | sg-05e93ed94c77b6f9c... | -           |

## Database and Table Creation

The following SQL commands were run via AWS CloudShell after connecting to the RDS instance using the PostgreSQL connz:

```

-- Connect to RDS from CloudShell
psql -h insurance-db.cql4g86akl23.us-east-1.rds.amazonaws.com \
-U postgres \
-d insuredadb

-- Customers table
CREATE TABLE customers (
    cust_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    risk_segment VARCHAR(20)
);

-- Policies table
CREATE TABLE policies (
    policy_id VARCHAR(20) PRIMARY KEY,
    cust_id INT,
    policy_type VARCHAR(50),
    premium_amount INT,
    FOREIGN KEY (cust_id) REFERENCES customers(cust_id)
);

-- Copy data of customers.csv to customers table
\copy customers(cust_id, name, email, risk_segment)
FROM 'customers.csv'
DELIMITER ','
CSV HEADER;

-- Copy data of policies.csv to policies table
\copy policies(policy_id, cust_id, policy_type, premium_amount)
FROM 'policies.csv'
DELIMITER ','
CSV HEADER;

-- Verify Tables
SELECT * FROM customers LIMIT 5;
SELECT * FROM policies LIMIT 5;

```

The CSV files were uploaded to CloudShell and loaded into the respective tables using LOAD DATA LOCAL INFILE commands. Data was verified using SELECT statements on both tables.

---

## 8. Amazon DynamoDB Setup

A DynamoDB table was created to store the claim lifecycle events and audit logs arriving from the source system.

| Parameter     | Value               |
|---------------|---------------------|
| Table Name    | ClaimStatusAudit    |
| Partition Key | claim_id (String)   |
| Sort Key      | updated_at (String) |

| Parameter      | Value            |
|----------------|------------------|
| Table Settings | Default settings |
| Status         | Active           |

The claims\_audit.json file was loaded into the DynamoDB table using AWS CloudShell commands. Item counts were verified through the DynamoDB console after the load completed.

The screenshot shows the AWS DynamoDB console interface for the 'ClaimStatusAudit' table. The table is active and contains 50 items. The console displays a list of items with columns: claim\_id (String), updated\_at (String), adjuster\_note, amount, and status. The status values include 'Under\_Review', 'Open', and 'Paid'.

| claim_id (String) | updated_at (String)      | adjuster_note   | amount  | status       |
|-------------------|--------------------------|-----------------|---------|--------------|
| CLM_1010          | 2026-04-26T14:46:13.8... | Requires review | 2422.47 | Under_Review |
| CLM_1010          | 2026-04-26T14:46:13.8... |                 | 2344.95 | Under_Review |
| CLM_1010          | 2026-04-27T19:46:13.8... |                 | 3622.79 | Under_Review |
| CLM_1010          | 2026-04-29T16:46:13.8... |                 | 1254.74 | Open         |
| CLM_1010          | 2026-04-30T02:46:13.8... |                 | 4501.7  | Open         |
| CLM_1010          | 2026-04-30T04:46:13.8... |                 | 2049.65 | Open         |
| CLM_1012          | 2026-04-26T06:46:13.8... |                 | 2186.87 | Under_Review |
| CLM_1012          | 2026-04-27T06:46:13.8... |                 | 2318.3  | Under_Review |
| CLM_1012          | 2026-04-27T20:46:13.8... |                 | 3479.55 | Under_Review |
| CLM_1012          | 2026-04-28T00:46:13.8... |                 | 3875.14 | Paid         |

## 9. AWS Glue ETL Job

A Glue job named Claims360-Job was created and attached with the Claims\_GlueETLRole IAM role. The job uses PySpark on Glue 4.0 with G.1X workers.

### Glue Job Configuration

| Parameter    | Value              |
|--------------|--------------------|
| Job Name     | Claims360-Job      |
| Glue Version | 4.0                |
| Worker Type  | G.1X               |
| Language     | Python (PySpark)   |
| IAM Role     | Claims_GlueETLRole |

ⓘ AWS Glue for Ray will no longer be open to new customers starting on April 30, 2026. If you would like to use the feature, please sign up prior to April 30, 2026. For capabilities similar to AWS Glue for Ray explore Amazon EKS. [Learn more here](#) ⓘ

## AWS Glue Studio Info

**Create job** Info

Author in a visual interface focused on data flow.  
**Visual ETL**

Author using an interactive code notebook.  
**Notebook**

Author code with a script editor.  
**Script editor**

▶ **Example jobs** Info Create example job

**Your jobs (1)** Info
⌂ Actions ▾ Run job

| <input type="checkbox"/> | Job name ▾                    | Type ▾   | Created by | Last modified         | AWS Glue version ▾ | Action ▾ |
|--------------------------|-------------------------------|----------|------------|-----------------------|--------------------|----------|
| <input type="checkbox"/> | <a href="#">Claims360_Job</a> | Glue ETL | Script     | 5/5/2026, 10:55:17 AM | 5.1                | -        |

## Extract Phase – Reading from Sources

Customers and Policies are read from Amazon RDS via JDBC, while Claim logs are read from DynamoDB using Glue's DynamicFrame API.

```
# — EXTRACT —
# 1. Customers
df_customers_raw = spark.read.format("jdbc").options(
    url="jdbc:postgresql://insurance-db.cq14g86akl23.us-east-1.rds.amazonaws.com:5432/insurance_db",
    dbtable="customers",
    user="postgres",
    password="virtusa12345",
    driver="org.postgresql.Driver"
).load()

df_customers = df_customers_raw \
    .withColumnRenamed("cust_id", "c_cust_id") \
    .withColumnRenamed("name", "c_name") \
    .withColumnRenamed("email", "c_email") \
    .withColumnRenamed("risk_segment", "risk_segment")

# 2. Policies
df_policies_raw = spark.read.format("jdbc").options(
    url="jdbc:postgresql://insurance-db.cq14g86akl23.us-east-1.rds.amazonaws.com:5432/insurance_db",
    dbtable="policies",
    user="postgres",
    password="virtusa12345",
    driver="org.postgresql.Driver"
).load()

df_policies = df_policies_raw \
    .withColumnRenamed("policy_id", "p_policy_id") \
    .withColumnRenamed("cust_id", "p_cust_id") \
    .withColumnRenamed("type", "policy_type") \
    .withColumnRenamed("premium", "premium_amount")
```

```
# 3. Claims from DynamoDB
df_claims_raw = glueContext.create_dynamic_frame.from_options(
    connection_type="dynamodb",
    connection_options={
        "dynamodb.input.tableName": "ClaimStatusAudit",
        "dynamodb.throughput.read.percent": "1.0",
        "dynamodb.region": "us-east-1"
    }
).toDF()
```

## Transform Phase

The transform phase handles type casting, schema evolution, deduplication, and key derivation to allow joining across the three data sources.

```
# Normalize column names
df_claims_raw = df_claims_raw.toDF(*[c.lower() for c in df_claims_raw.columns])
# -----
# SCHEMA EVOLUTION HANDLING
# -----
if "adjuster_note" not in df_claims_raw.columns:
    df_claims_raw = df_claims_raw.withColumn("adjuster_note", lit(None))
# Handle schema differences
if "amount" in df_claims_raw.columns:
    df_claims_raw = df_claims_raw.withColumnRenamed("amount", "amount_claimed")
# Fix types
df_claims_raw = df_claims_raw.withColumn(
    "updated_at", to_timestamp(col("updated_at"))
).withColumn(
    "amount_claimed", col("amount_claimed").cast("double")
)
# Add derived keys
df_claims = df_claims_raw \
    .withColumn(
        "derived_cust_id",
        (regexp_extract(col("claim_id"), r"(\d+)", 1).cast("int") % 50 + 1)
    ) \
    .withColumn(
        "derived_policy_id",
        concat(
            lit("POL_"),
            (
```

```

        (regexp_extract(col("claim_id"), r"(\d+)", 1).cast("int") % 50 + 1) + 100
    ).cast("string")
)
)
# --- TRANSFORM -----

df_step1 = df_claims.join(
    df_policies,
    df_claims["derived_policy_id"] == df_policies["p_policy_id"],
    "left"
)

df_step2 = df_step1.join(
    df_customers,
    df_step1["derived_cust_id"] == df_customers["c_cust_id"],
    "left"
)

df_final_raw = df_step2.select(
    col("claim_id"),
    col("updated_at"),
    col("status"),
    col("amount_claimed").alias("amount"),
    col("derived_cust_id").alias("cust_id"),
    col("derived_policy_id").alias("policy_id"),
    col("policy_type"),
    col("premium_amount"),
    col("c_name").alias("customer_name"),
    col("risk_segment"),
    col("adjuster_note") # ✅ NEW COLUMN
)

# Latest record per claim
window_spec = Window.partitionBy("claim_id").orderBy(col("updated_at").desc())

df_final = df_final_raw \
    .withColumn("row_num", row_number().over(window_spec)) \
    .filter(col("row_num") == 1) \
    .drop("row_num")

print(f"Total records to write: {df_final.count()}")
df_final.printSchema()

```

## Load Phase – Writing to Delta Lake

The transformed data is written to the Silver layer in Delta format. On the first run an initial write is performed; on subsequent runs a MERGE (upsert) operation is applied to keep the current state table up to date. All historical records are appended separately to the history table.

```
# — LOAD (DELTA MERGE) —————
if DeltaTable.isDeltaTable(spark, SILVER_PATH):
    delta_table = DeltaTable.forPath(spark, SILVER_PATH)

    delta_table.alias("existing").merge(
        df_final.alias("incoming"),
        "existing.claim_id = incoming.claim_id"
    ).whenMatchedUpdateAll() \
      .whenNotMatchedInsertAll() \
      .execute()

    print("Merge complete.")
else:
    df_final.write.format("delta") \
      .mode("overwrite") \
      .option("mergeSchema", "true") \
      .save(SILVER_PATH)

    print("Initial write complete.")

# — DONE —————
job.commit()
print("✓ ETL job finished successfully.")
```

---

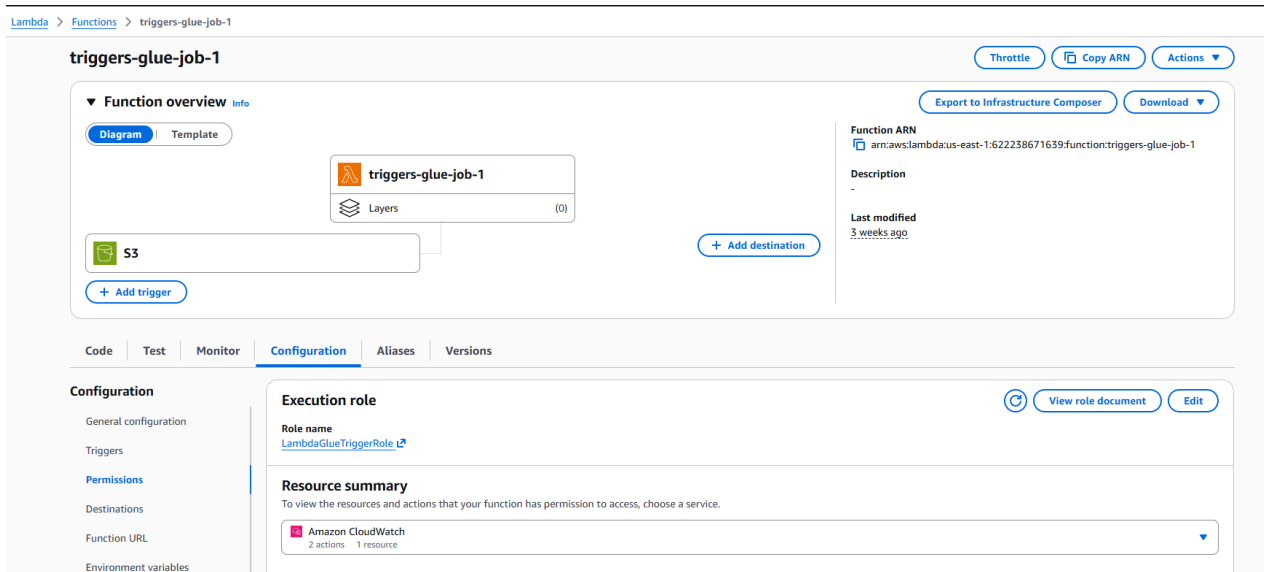
## 10. AWS Lambda Trigger Setup

A Lambda function named triggers-glue-job-1 was created to automatically initiate the Glue ETL job whenever new data files arrive in the S3 Bronze layer.

### S3 Event Trigger Configuration

| Parameter         | Value                            |
|-------------------|----------------------------------|
| Bucket ARN        | arn:aws:s3:::claims360-lakehouse |
| Event Type        | s3:ObjectCreated:Put             |
| Prefix            | bronze/rds/                      |
| Suffix            | policies.csv                     |
| Notification Name | lambda-glue-trigger              |

The trigger fires when policies.csv is uploaded to the bronze/rds/ path, which marks completion of a data batch. The Lambda function then starts the Glue job programmatically.



## Lambda Function Code

```
import boto3
glue = boto3.client('glue')
def lambda_handler(event, context):
    print("Event received:", event)
    # SAFE CHECK
    if "Records" not in event:
        print("No S3 event found")
        return {"statusCode": 200}
    for record in event["Records"]:
        key = record["s3"]["object"]["key"]
        print("File uploaded:", key)
        if "policies.csv" in key:
            try:
                response = glue.start_job_run(
                    JobName="Claims360_Job"
                )
                print("Glue Job Started:", response["JobRunId"])
            except Exception as e:
                print("Glue start failed:", str(e))
    return {"statusCode": 200}
```

## End-to-End Pipeline Flow

Once the policies.csv file lands in the Bronze layer, the pipeline executes in the following sequence:

| Step | Action                                                                                       |
|------|----------------------------------------------------------------------------------------------|
| 1    | CSV files (customers.csv, policies.csv) are uploaded to s3://claims360-lakehouse/bronze/rds/ |

| Step | Action                                                                     |
|------|----------------------------------------------------------------------------|
| 2    | claims_audit.json is uploaded to s3://claims360-lakehouse/bronze/dynamodb/ |
| 3    | S3 ObjectCreated event fires and invokes the Lambda function               |
| 4    | Lambda calls glue.start_job_run() to trigger the Claims360-Job             |
| 5    | Glue reads from RDS and DynamoDB, transforms and joins the data            |
| 6    | Delta Lake tables are written or merged in the Silver layer                |
| 7    | Parquet files and Delta transaction logs appear in the target S3 location  |

## 11. Query Using Amazon Athena

The Delta table was registered in the AWS Glue Data Catalog so it can be queried using Amazon Athena. The table was created using the following DDL:

```
CREATE EXTERNAL TABLE claims_table_v2
LOCATION 's3://claims360-lakehouse/silver/delta/'
TBLPROPERTIES ('table_type' = 'DELTA');
```

The screenshot displays the Amazon Athena console interface. On the left, the 'Data' sidebar shows the 'claims-insurance-db' database and the 'claims\_table\_v2' table. The main area shows the SQL query editor with the query: `SELECT * FROM 'claims-insurance-db'.claims_table_v2' limit 10;`. Below the editor, the 'Query results' section shows the execution status as 'Completed' with a run time of 1.857 sec and 1.37 KB of data scanned. The results are displayed as a table with 10 rows and 12 columns.

| # | claim_id | updated_at                  | status | amount  | cust_id | policy_id | policy_type | premium_amount | customer_name | risk_segment | adjuster_note |
|---|----------|-----------------------------|--------|---------|---------|-----------|-------------|----------------|---------------|--------------|---------------|
| 1 | CLM_1000 | 2026-04-30 07:46:13.886 UTC | Open   | 1298.9  | 1       | POL_101   | Auto        | 1675.00        | Insured_1     | Medium       |               |
| 2 | CLM_1003 | 2026-04-28 01:46:13.887 UTC | Paid   | 4262.12 | 4       | POL_104   | Auto        | 920.00         | Insured_4     | Medium       |               |
| 3 | CLM_1004 | 2026-04-30 08:46:13.886 UTC | Open   | 4090.85 | 5       | POL_105   | Auto        | 811.00         | Insured_5     | Medium       |               |
| 4 | CLM_1005 | 2026-04-29 22:46:13.886 UTC | Paid   | 1632.49 | 6       | POL_106   | Auto        | 1967.00        | Insured_6     | Low          |               |
| 5 | CLM_1006 | 2026-04-29 13:46:13.886 UTC | Open   | 3060.93 | 7       | POL_107   | Auto        | 1485.00        | Insured_7     | Medium       |               |
| 6 | CLM_1001 | 2026-04-29 14:46:13.886 UTC | Open   | 2640.84 | 2       | POL_102   | Auto        | 593.00         | Insured_2     | Medium       |               |
| 7 | CLM_1002 | 2026-04-29 22:46:13.886 UTC | Open   | 1366.53 | 3       | POL_103   | Auto        | 1929.00        | Insured_3     | Medium       |               |

## Business Query – Total Paid Claims per Risk Segment

```
SELECT
    risk_segment,
    SUM(amount) AS total_paid_claims,
    COUNT(claim_id) AS claim_count
FROM claims_table
WHERE status = 'Paid'
GROUP BY risk_segment;
```

The query aggregates all claims with a Paid status and groups them by the customer's risk segment, giving analysts a quick view of financial exposure across risk categories.

Query results

Query stats

Completed

Time in queue: 106 msRun time: 1.316 secData scanned: 0.53 KB

Results (3)

Copy

Download results CSV

Search rows

| # | risk_segment | total_paid_claims | claim_count |
|---|--------------|-------------------|-------------|
| 1 | Low          | 9205.89           | 4           |
| 2 | High         | 8614.04           | 2           |
| 3 | Medium       | 8824.73           | 2           |

## 12. Time Travel Query

Delta Lake transaction log stored in S3 `_delta_log/` folder — each JSON file represents one committed version, enabling time travel and ACID compliance.

**\_delta\_log/**
Copy S3 URI

---

**Objects**
Properties

**Objects (2/5)** 
[Copy S3 URI](#)
[Copy URL](#)
[Download](#)
[Open ↗](#)
[Delete](#)
[Actions ▼](#)
[Create folder](#)
[Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more ↗](#)

Find objects by prefix

| <input type="checkbox"/>            | Name ▲                                            | Type ▼ | Last modified ▼                         | Size ▼ | Storage class ▼ |
|-------------------------------------|---------------------------------------------------|--------|-----------------------------------------|--------|-----------------|
| <input type="checkbox"/>            | _commits/                                         | Folder | -                                       | -      | -               |
| <input checked="" type="checkbox"/> | <a href="#">0000000000000000000000000000.crc</a>  | crc    | May 4, 2026,<br>15:12:21<br>(UTC+05:30) | 2.4 KB | Standard        |
| <input type="checkbox"/>            | <a href="#">0000000000000000000000000000.json</a> | json   | May 4, 2026,<br>15:12:12<br>(UTC+05:30) | 2.6 KB | Standard        |
| <input checked="" type="checkbox"/> | <a href="#">0000000000000000000000000000.crc</a>  | crc    | May 5, 2026,<br>11:01:18<br>(UTC+05:30) | 1.5 KB | Standard        |
| <input type="checkbox"/>            | <a href="#">0000000000000000000000000000.json</a> | json   | May 5, 2026,<br>11:01:17<br>(UTC+05:30) | 3.6 KB | Standard        |

## 13. Schema Evolution and ACID Compliance

## Schema Evolution

A new column `adjuster_note` was added to the DynamoDB source to demonstrate Delta Lake's schema evolution capability. The Glue script handles this gracefully by checking for the column's existence at runtime and inserting a null placeholder if it is absent. The `mergeSchema` option ensures that the Delta table schema is extended automatically without breaking existing queries.

```
# Handle schema evolution at runtime
if "adjuster_note" not in df_claims_raw.columns:
    df_claims_raw = df_claims_raw.withColumn("adjuster_note", lit(None))

# Enable automatic schema merging on Delta writes
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
```

| claims_table_v2 |               |   |
|-----------------|---------------|---|
| claim_id        | string        | ⋮ |
| updated_at      | timestamp     | ⋮ |
| status          | string        | ⋮ |
| amount          | double        | ⋮ |
| cust_id         | int           | ⋮ |
| policy_id       | string        | ⋮ |
| policy_type     | string        | ⋮ |
| premium_amount  | decimal(10,2) | ⋮ |
| customer_name   | string        | ⋮ |
| risk_segment    | string        | ⋮ |
| adjuster_note   | string        | ⋮ |

The adjuster\_note column has been added in the external table claims\_table\_v2.

```
SELECT
  claim_id,
  adjuster_note
FROM "claims-insurance-db"."claims_table_v2"
```

| Query results |          | Query stats           |                                      |
|---------------|----------|-----------------------|--------------------------------------|
| Completed     |          | Time in queue: 107 ms | Run time: 1.834 sec                  |
|               |          | Data scanned: 0.16 KB |                                      |
| Results (21)  |          | <a href="#">Copy</a>  | <a href="#">Download results CSV</a> |
| Search rows   |          | < 1 > ⚙               |                                      |
| #             | claim_id | adjuster_note         |                                      |
| 1             | CLM_1000 |                       |                                      |
| 2             | CLM_1001 |                       |                                      |
| 3             | CLM_1002 |                       |                                      |
| 4             | CLM_1003 |                       |                                      |
| 5             | CLM_1004 |                       |                                      |
| 6             | CLM_1005 |                       |                                      |
| 7             | CLM_1006 |                       |                                      |
| 8             | CLM_1007 |                       |                                      |
| 9             | CLM_1008 |                       |                                      |
| 10            | CLM_1009 |                       |                                      |

## ACID Compliance

| Property    | How It Is Achieved                                                                                                                                              |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Atomicity   | Delta transaction log prevents partial commits. If the Glue job fails midway, the table remains in its last consistent state with no corrupted records visible. |
| Consistency | All schema constraints and data types remain valid after every write.                                                                                           |
| Isolation   | Concurrent reads and writes are safely managed through Delta's optimistic concurrency control.                                                                  |
| Durability  | All committed data is permanently stored in Amazon S3 and referenced by immutable transaction log entries.                                                      |

---

## 14. Challenges and Resolutions

| Challenge                                        | Resolution                                                                        |
|--------------------------------------------------|-----------------------------------------------------------------------------------|
| Joining relational and NoSQL data sources        | Used Glue DynamicFrames for DynamoDB reads combined with Spark joins              |
| Handling repeated claim IDs across audit entries | Implemented a Window Function to select the latest record per claim_id            |
| Managing schema changes from the source          | Enabled Delta mergeSchema and handled missing columns at runtime                  |
| Ensuring ACID-compliant writes                   | Leveraged the Delta Lake transaction log for atomic merge operations              |
| Querying historical claim data                   | Built a separate history table and used Athena time-travel queries                |
| Automating pipeline execution                    | Configured an S3 event notification to invoke Lambda, which triggers the Glue job |

---

## 15. Business Benefits

| Benefit                  | Description                                                            |
|--------------------------|------------------------------------------------------------------------|
| Centralized Analytics    | Unified view of customer, policy, and claims data in a single platform |
| Historical Tracking      | Full claim lifecycle visibility from Open to Paid or Denied            |
| Improved Decision Making | Better foundation for fraud detection and risk analysis                |
| Scalability              | Fully serverless and scalable AWS architecture                         |

| Benefit           | Description                                                              |
|-------------------|--------------------------------------------------------------------------|
| Cost Optimization | Pay-as-you-go pricing across S3, Glue, Lambda, and Athena                |
| Faster Querying   | Athena provides serverless SQL analytics with no infrastructure overhead |
| Data Reliability  | ACID-compliant Delta Lake ensures trustworthy, consistent data           |

---

## 16. Future Enhancements

The following capabilities are planned for the next phase of the project:

- Real-time streaming ingestion using Amazon Kinesis
  - Fraud detection models using Amazon SageMaker
  - Workflow orchestration using AWS Step Functions
  - Data quality validation using AWS Glue DataBrew
  - Business dashboard integration with Amazon QuickSight
  - Incremental change data capture using DynamoDB Streams
  - CI/CD pipeline deployment using AWS CodePipeline
- 

## Appendix – AWS Services Summary

| Service                 | Purpose                                        |
|-------------------------|------------------------------------------------|
| Amazon S3               | Data Lake storage for Bronze and Silver layers |
| Amazon RDS (PostgreSQL) | Relational source for customers and policies   |
| Amazon DynamoDB         | NoSQL source for claim audit logs              |
| AWS Glue                | PySpark-based distributed ETL engine           |
| AWS Lambda              | Event-driven pipeline automation               |
| Amazon Athena           | Serverless SQL analytics over Delta tables     |
| AWS IAM                 | Security and least-privilege access control    |
| Amazon CloudWatch       | Job monitoring and logging                     |
| Delta Lake              | ACID-compliant Lakehouse storage format        |
| AWS Glue Data Catalog   | Metadata repository for table registration     |